Lo que queda y lo que ya está

LudusERP · plan de los dos últimos sprints, por orden de prioridad

Fecha: 12 de noviembre de 2026 **Código:** master en `d81e07c` **Pendientes:** 7 **Realizados:** 6

Cómo leerlo

Los apartados pendientes van primero y en el orden en que conviene atacarlos: los de arriba desbloquean a los de abajo. Cada uno lleva una etiqueta — IMPRESCINDIBLE no se puede entregar sin ello, IMPORTANTE se nota si falta, SI SOBRA TIEMPO es deuda que no bloquea la entrega. Lo que ya está terminado lleva REALIZADO y se ha movido al final, en verde. **Los números no se han cambiado al reordenar**: el apartado 8 sigue siendo el 8 aunque ahora aparezca después del 15, para que las referencias de la memoria y de los commits sigan valiendo. Los antiguos apartados 4 y 5 — los datos de prueba de pedidos y el test de contexto — se han **descartado**: no se van a hacer, y por eso ya no aparecen. **El sprint 1 queda cerrado**.

Ramas

Una rama corta por apartado, salida de master y mergeada en cuanto el apartado funcione. Es lo que se ha hecho con `tienda-cliente`, `juegos-colaborador` y `creacion-de-valoraciones`, y ha funcionado: cada merge ha dejado master arrancable.

El camino crítico

Era la cadena **6 → 7 → 8**: de un colaborador que sube su juego a un cliente que lo compra, recibe su clave y se lo descarga. **El 6 y el 8 ya están**, así que hoy solo falta el eslabón del medio: **el 7**, la cesta y el pago. Mientras no exista, no hay forma de que nazca una clave en la aplicación y las que hay son las que siembra `DatosIniciales`.

SPRINT 2 · CERRAR LA VENTA Y ENTREGAR

7 Cesta y pago con Stripe

IMPRESCINDIBLE

> Montar la cesta del cliente y el pago con Stripe, que al confirmarse convierte la cesta en pedido y factura, y acuña la clave.

Es el apartado más grande que queda, el que el profesor pidió expresamente y **el único eslabón que falta del camino crítico**. Ojo, que el punto de partida es mejor de lo que parece: `CestaDao` y `CestaServiceImpl` **ya existen**, con su `@Transactional` y comprobando que la línea sea de la cesta de quien la toca. Lo que no existe es el otro lado: `ControllerCesta` está creado pero **vacío, sin una sola ruta**, no hay plantilla de cesta, y en la ficha el botón sigue deshabilitado con el aviso de que la cesta no existe. Falta también la dependencia de Stripe, que no está en el pom, crear la sesión de pago y recoger la confirmación en un webhook o en la vuelta del pago para marcar el pedido como PAGADO. Los datos de tarjeta los teclea el cliente en Stripe, nunca en un formulario nuestro.

Ojo con tres cosas que ya sabemos: una línea de cesta con cantidad 2 se expande en dos líneas de pedido, porque una línea de pedido es una clave; la factura tiene que guardar copia del nombre y la dirección, o el día que un cliente se mude le cambiarán las facturas viejas; y al confirmar el pago es donde se acuña la clave, con su `codigo_hash` relleno a mano con `IndiceCiego.hash(valor)`, que no se calcula solo.

Rama recomendada: `cesta-y-pago-stripe`

> Los juegos gratis **no pasan por aquí**: se descargan directamente desde su ficha y no llevan clave. La cesta y el pago son solo para los de pago.

9 Órdenes de compra y recepción de claves

IMPORTANTE

> Dar de alta órdenes de compra a un proveedor y recibir las claves que entran por cada línea.

Es la mitad izquierda del esquema: proveedor → compra → línea de compra → clave. `Compra` y `LineaCompra` existen como entidades, pero no las usa nadie: no hay DAO ni CRUD. El CRUD de proveedores ya está hecho y sirve de punto de partida. Dos reglas que el modelo ya lleva dentro: cantidad nula significa ilimitada, y el empleado que firma la compra es obligatorio.

Rama recomendada: `compras-y-recepcion`

> Es lo primero que se sacrifica si el 7 se come el sprint: sin esto el ERP sigue enseñándose entero por el lado de la venta. Las diez claves que hay hoy en la base están sembradas a mano y **ninguna** tiene línea de compra detrás, que es exactamente lo que este apartado arregla.

10 Notificaciones en tiempo real y tareas en segundo plano NUEVO IMPRESCINDIBLE

> Avisar por WebSocket a quien tenga el panel abierto, y sacar de la petición el trabajo que no tiene por qué hacer esperar al usuario.

Es el **requisito 3.5** y el pom lo lleva desde el primer día: `spring-boot-starter-websocket` y su starter de prueba están declarados y **no hay ni una línea que los use**. Lo mismo con los hilos: la pantalla `admin/jobs` del prototipo enseña seis procesos con `@Async` y `@Scheduled`, y en el código no aparece ninguna de las dos anotaciones.

Los hilos. Una configuración con `@EnableAsync` y un `@Async` en el envío de correo: hoy `EmailServiceImpl` es síncrono, así que el gestor que activa a un cliente se queda esperando a que conteste el servidor de correo. Y un `@Scheduled` para la purga, que no es un adorno: en cuanto el apartado 7 reserve claves al iniciar el pago, una clave reservada cuyo pago no se termina queda bloqueada **para siempre** si nadie la libera. `ClaveLicencia.fechaReserva` existe justo para eso.

Los sockets. Un `WebSocketConfig` con `@EnableWebSocketMessageBroker`, un broker simple sobre `/topic` y el endpoint de conexión. Con dos avisos basta para justificar el requisito: el de **stock bajo**, cuando las claves disponibles de un juego caen por debajo de su `umbral_stock_bajo`, una columna que ya existe y que hoy no lee nadie, y el de **pedido nuevo** para el gestor de clientes. En el navegador, un script que se suscribe y pinta el aviso en la barra del panel.

Rama recomendada: `notificaciones-tiempo-real`

> El canal también tiene rol: un cliente no se puede suscribir a `/topic/inventario`. Las reglas de `SecurityConfig` son por URL y no cubren el socket, así que eso se configura aparte. El aviso de stock bajo ya tiene de dónde leer, porque `clave_licencia` tiene filas; la purga depende del 7.

12 Arreglos sueltos del panel IMPORTANTE

> Los detalles que quedan, los que no rompen nada pero un profesor encuentra en dos clics.

Al listado de clientes le falta el botón **Limpiar**, y a los de departamentos y facturas también. La paginación de `ventas/pedidos` y `ventas/facturas` sigue siendo decorativa, con las flechas deshabilitadas y un 1 fijo escritos a mano en la plantilla, mientras los otros nueve listados ya la tienen real. Y el filtro de facturas no tiene botón de enviar: solo funciona pulsando Enter.

Rama recomendada: `arreglos-panel`

13 Dejarlo listo para producción

IMPRESCINDIBLE

> Sacar el perfil dev de `application.properties`, pasar los secretos al entorno y desplegar la aplicación en algún sitio.

La aplicación tiene que acabar en producción porque los juegos del tercer proyecto se publicarán en ella. Hoy `spring.profiles.active=dev` está fijado en el fichero, así que un despliegue tal cual cargaría los usuarios de prueba con contraseña conocida, uno de ellos ADMIN, y ahora también diez claves de licencia inventadas. El perfil tiene que venir del entorno, igual que `LUDUSERP_AES_KEY`, y conviene separar esa clave de la del índice ciego, que ahora es la misma. Y hay que resolver dónde viven los archivos subidos: `app.uploads.dir` y `app.juegos.dir` apuntan a carpetas relativas al proyecto, y en un hosting que recrea el disco en cada despliegue se lleva por delante las imágenes **y los ZIP de los juegos**.

Rama recomendada: `preparar-produccion`

14 Documentación de entrega: PDF, prototipo y memoria

IMPRESCINDIBLE

> Rehacer el documento de diseño para que cuente lo que de verdad se ha construido, y volver a sincronizar prototipo y memoria.

El PDF va con más de veinte correcciones pendientes apuntadas en la memoria: sobra la clase Desenvolupador y las tablas de permisos, falta el actor COLABORADOR entero, la jerarquía de Usuari abstracta, los siete roles en vez de cinco, Stripe en lugar del pago simulado, y Categoria y Plataforma como clases. Además promete cosas que no se usan (MapStruct, Bootstrap, nombres en inglés) y dice Spring Boot 3 cuando el pom es 4.1.1. A esa lista se suman las **valoraciones** del apartado 11 y las **notificaciones** del 10.

La memoria (`docs/memoriaClaudeByAleix.txt`) se quedó en el 4 de noviembre, y desde entonces no recoge ventas, correo, paginación, el CRUD de videojuegos, las imágenes, las fotos de perfil, la tienda, Colaboraciones, la biblioteca, la descarga ni las valoraciones. Además su decisión **2.6** ya no es cierta: decía que la clave es la prueba de titularidad «también cuando el juego es gratis», y ahora los juegos gratis no llevan clave.

El prototipo tiene el mismo problema en una sola pantalla: `tienda/biblioteca` dice «la clave es la prueba de que el juego es tuyo, y es lo que deja descargarlo» y le pinta un código a un juego gratis. Hay que rehacer ese texto y quitar ese código.

Rama recomendada: `documentacion-entrega`

> Es trabajo de varias horas y no se puede dejar para el último día, porque es la mitad de lo que se corrige.

15 Las chapuzas conocidas

SI SOBRA TIEMPO

> Barrer la deuda pequeña que se viene apuntando desde octubre.

El `@Transactional` ya lo llevan clientes, departamentos, cesta, videojuegos y biblioteca, pero el resto de servicios no, así que validar y guardar no es atómico: dos altas simultáneas con el mismo NIF pasan las dos la comprobación y la segunda muere con un 500 en vez del mensaje. `listarPortadas` está copiado **tres veces**, en los servicios de tienda, de videojuegos y de biblioteca. No hay `mvnw`. El healthcheck de MySQL de `compose.yaml` debería ser CMD-SHELL. `nbactions.xml` arrastra `-noverify`, obsoleta desde JDK 13. Y el pom sigue con artifactId `demo` y groupId `com.example`.

Rama recomendada: `deuda-pequena`

YA REALIZADO · 6 APARTADOS

1 Punto de control: recrear la base, arrancar y mergear la tienda

REALIZADO

> La base se recreó, la aplicación arranca y las ramas están mergeadas.

Se hizo el `docker compose down -v`, Flyway aplicó las migraciones sobre una base limpia y Hibernate aceptó el esquema con `ddl-auto=validate`. Hoy valida **diez** migraciones, porque la V10 de valoraciones entró el 10 de noviembre. `tienda-cliente`, `juegos-colaborador` y `creacion-de-valoraciones` están mergeadas en master.

Lo que se probó por primera vez en ejecución salió bien: la tienda entera con sus dos `@Query` de `coalesce` y parámetros de tres estados, las imágenes de juego, las fotos de perfil, que un colaborador entra y cae en la tienda, que un gestor de inventario abre Ventas, que un cliente que escriba `/ventas/pedidos` se come un 403 y que cerrar sesión cierra de verdad.

> **De paso quedó esto:** al recrear la base, las fotos de perfil y las imágenes de juego que había subidas se quedaron en `uploads/` sin ninguna fila que las apunte. Archivos huérfanos, inofensivos, pero ahí siguen.

2 Cerrar las decisiones que bloquean código

REALIZADO

> Las cuatro decisiones están cerradas, y además escritas en código, no solo acordadas.

Dónde vive el ZIP: en `juegos-privados/<id del juego>/`, con nombre UUID, **fuera** de `uploads/`, porque todo lo que cuelga de ahí lo sirve Spring como recurso y cualquiera se lo podría bajar. Lo sirve un controlador nuestro. **Un juego de colaborador nace inactivo** y alguien lo tiene que publicar. **Todos sus juegos son gratis**: el servicio fuerza `precio = 0` al subirlos. Y **no conviven** con los de distribuidor en el catálogo: se les hizo su propia pantalla, Colaboraciones, al lado de Ofertas.

Después se cerró una quinta que no estaba en esta lista y que cambia el apartado 8: **un juego gratis no lleva clave de licencia**. Se descarga directamente desde su ficha y no entra en la biblioteca, porque la biblioteca existe para no perder lo que pagaste y un juego gratis no se puede perder: sigue en la tienda.

> **Queda apuntarlo en la memoria**, que es parte del apartado 14. Y la quinta decisión invalida la 2.6 de la memoria, que decía justo lo contrario.

3 Arreglar las dos pantallas que hoy están rotas

REALIZADO

> Las dos arregladas y comprobadas sirviendo la página.

El detalle de pedido ya no pinta `l.videojuego.estudio`, el campo que la decisión 4.15 sustituyó por la clave ajena al proveedor; era la última referencia a ese campo en todo el código. **«Mis datos»** tiene ya su `th:action`, su `method` y el POST en `ControllerHome`, con el guardado colgado de `UsuarioService`: lo que escribes se guarda y sale su aviso.

6 El colaborador sube sus juegos

REALIZADO

> La pantalla existe, funciona y se ha probado subiendo un juego de verdad.

Se crearon `ControllerColaboradorJuegos`, el `publicarJuegoColaborador` de `VideojuegoServiceImpl`, `VersionJuegoDao`, las plantillas `colaboradores/mis-juegos.html` y `colaboradores/nuevo-juego.html`, y `colaboradorJuegos.css`. El ZIP no se cree por la extensión: se comprueba su firma `PK\3\4`, se guarda con nombre UUID y se calcula su SHA-256 mientras se copia, sin leer el archivo dos veces. El juego queda con `colaborador_id`, sin proveedor, gratis, inactivo y con stock ilimitado, y su ZIP apuntado en `version_juego`. Si algo falla a mitad, se deshacen las filas y se borran los archivos.

Probado de punta a punta: subida correcta, un ZIP falso rechazado con su mensaje y el formulario devuelto con lo que ya estaba escrito.

> **Lo que quedó fuera: la contraseña del colaborador**. Sigue sin haber ninguna vía, ni para el admin ni para él mismo: el servicio lo soporta, pero `admin/formularioEditarCol.html` no ofrece el campo. Son seis líneas, las mismas que ya tiene el formulario de empleado. Y dos detalles: el mensaje de éxito promete una revisión que no existe, y no hay ninguna pantalla donde el gestor vea los juegos pendientes de publicar — hay que buscarlos filtrando por inactivo en el CRUD de videojuegos.

8 Claves de licencia, biblioteca y descarga

REALIZADO

> La biblioteca enseña tus claves y la descarga sirve el ZIP solo a quien le toca.

La biblioteca no es una tabla nueva: es una consulta. Se llegó a plantear una tabla propia para poder guardar propiedad sin clave, y se descartó al decidir que los juegos gratis no entran en la biblioteca. Así que son las claves VENDIDAS del cliente, que es justo lo que decía la decisión 7.4 de la memoria, y no hizo falta ninguna migración. Se crearon `ClaveLicenciaDao`, `BibliotecaService` y su impl, `ControllerBiblioteca`, `client/biblioteca.html` y dos excepciones propias.

La descarga la sirve `/tienda/descargar/{id}`, y el permiso lo decide el servicio con dos motivos para dejarte pasar: o el juego es gratis y está publicado, o tienes una clave VENDIDA suya. El email sale siempre de la sesión y nunca de la URL. `AlmacenArchivos.leerZip` comprueba que la ruta guardada sigue cayendo dentro de `juegos-privados`, y el archivo se sirve a trozos, nunca entero en memoria. `DatosIniciales` siembra diez claves: seis VENDIDAS repartidas entre tres clientes, tres DISPONIBLE y una ANULADA, que son las que demuestran que el filtro de la biblioteca es correcto.

Probado: el ZIP se descarga con el md5 idéntico al subido, un cliente no ve ni se baja lo de otro, un juego sin publicar no se descarga y un juego de pago sin clave responde 403.

> **Lo que quedó fuera: acuñar la clave al pagar**, que es del apartado 7 y no se puede hacer sin él. Hasta entonces, las únicas claves de la base son las sembradas. Y queda sin probar el caso «juego de pago, con clave y con ZIP», porque ninguno de los juegos sembrados tiene archivo.

11 Valoraciones de los juegos

NUEVO REALIZADO

> Un cliente puntúa y comenta un juego, y las reseñas salen en su ficha.

Llegó con la **V10**, la única migración nueva de todo el plan: tabla `valoracion` con el cliente, el juego, el estado, el título, la descripción y la puntuación, con un `CHECK` que obliga a que esté entre 1 y 5. Con ella, la entidad `Valoracion` y su `EstadoValoracion`, `ValoracionDao`, el servicio, `ControllerValoracion` y, en la ficha, el selector de estrellas y la lista de reseñas debajo. Solo se listan las valoraciones ACTIVAS de clientes que siguen activos, y la consulta trae el cliente con `join fetch` porque `open-in-view` está desactivado.

> **Lo que quedó fuera**, y conviene cerrarlo antes de entregar: **no se comprueba que el cliente tenga el juego**, así que hoy cualquier cliente puede valorar cualquier juego, cuando la regla era que solo valora quien tiene una clave VENDIDA suya. Falta el `UNIQUE (videojuego_id, cliente_id)` en la tabla, que era lo que debía garantizar una sola valoración por cliente y juego: ahora se comprueba a mano en el servicio. La tabla **no guarda la fecha** de la valoración. Y **la nota media no sale** ni en la ficha ni en la tarjeta del catálogo: no hay ningún `avg()` y `TiendaServiceImpl` no la lleva.

Resumen

#	APARTADO	RAMA	SPRINT	ESTADO
7	Cesta y pago con Stripe	`cesta-y-pago-stripe`	2	Imprescindible
9	Órdenes de compra y recepción	`compras-y-recepcion`	2	Importante
10	Notificaciones en tiempo real y tareas en segundo plano	`notificaciones-tiempo-real`	2	Imprescindible
12	Arreglos sueltos del panel	`arreglos-panel`	2	Importante
13	Dejarlo listo para producción	`preparar-produccion`	2	Imprescindible
14	Documentación de entrega	`documentacion-entrega`	1 y 2	Imprescindible
15	Las chapuzas conocidas	`deuda-pequena`	2	Si sobra tiempo
1	Punto de control: recrear la base, arrancar y mergear	`punto-control-esquema`	1	**Realizado**
2	Cerrar las decisiones abiertas	`decisiones-pendientes`	1	**Realizado**
3	Arreglar las dos pantallas rotas	`arreglos-rotos`	1	**Realizado**
6	El colaborador sube sus juegos	`juegos-colaborador`	1	**Realizado**
8	Claves, biblioteca y descarga	`biblioteca-y-claves`	2	**Realizado**
11	Valoraciones de los juegos	`valoraciones-de-juegos`	2	**Realizado**

> Quedan siete apartados, todos del sprint 2. Si el tiempo se tuerce, el orden de recorte es: primero el 15, luego el 9 y por último el 12. Lo que no se puede recortar es **el 7**, que es el único eslabón que falta para cerrar la cadena 6 → 7 → 8 y el que pidió el profesor; ni el 10, que es requisito de la asignatura; ni el 13 ni el 14, porque sin ellos no hay ni aplicación en producción ni documento que entregar. Sin test de contexto, cada apartado que se cierre hay que seguir verificándolo a mano: arrancar y pedir las pantallas una por una.